

AMBA APB Subsystem

Design Documentation

APB master controller, address decoder and two memory-mapped slave controllers

Table of Contents

Table of Contents.....	2
1. Introduction	5
1.1 Project Overview.....	5
1.2 Objectives.....	5
1.3 Scope.....	5
1.4 Tools and Environment	5
1.5 Definitions and Abbreviations.....	5
2. AMBA APB Protocol Background	6
2.1 Overview of AMBA APB	6
2.2 Protocol Signals.....	6
2.3 Transfer Phases.....	7
2.4 Write and Read Transfers	7
2.5 Byte Strokes and Protection (APB4)	7
3. Requirements and Specifications.....	7
3.1 Functional Requirements.....	7
3.2 Design Parameters	8
3.3 Assumptions and Constraints	8
4. System Architecture.....	8
4.1 Top-Level Block Diagram.....	8
4.2 Module Hierarchy	9
4.3 Data and Control Flow	9
4.4 Memory Map	10
4.5 Top-Level Interface	10
Clock, Reset and System Request Interface.....	10
System Response Interface.....	10
5. Detailed Design	11
5.1 APB Master Controller	11
Purpose	11
Parameters.....	11
Port List	11
FSM Description	12
State Transition Table	12
Single-Transfer Timing	13
Functional Description	13
Output Register Behavior.....	14
Back-to-Back Transfer Handling.....	14

Source Code	14
5.2 APB Address Decoder	17
Purpose	17
Parameters.....	18
Port List	18
Address Ranges	18
Functional Description	18
Block Diagram / Waveform	18
Source Code	19
5.3 APB Slave Controller	19
Purpose	19
Parameters.....	19
Port List	20
Internal Memory	20
Byte-Strobe Write Behavior	20
Error Generation	20
Wait States	20
Read Path Timing	20
Block Diagram / Waveform	21
Source Code	21
5.4 Top Level: Select Gating and Response Multiplexer	22
Purpose	22
Select Gating	22
Response Multiplexer	23
Default Behavior When No Slave Is Selected.....	23
Parameter Propagation.....	23
Source Code	23
6. Verification.....	27
6.1 Verification Strategy	27
6.2 Testbench Architecture.....	27
6.3 Test Plan.....	27
6.4 Test Case Details	27
Test 1: Full-Word Access	27
Test 2: Byte-Strobe Masking	28
Test 3: Out-of-Bounds Error Detection	28
Test 4: Back-to-Back Transfers.....	28
6.5 Coverage	28
7. Simulation	29

7.1 How to Run	29
7.2 Waveform Results.....	30
7.3 Console Log Output.....	30
8. Conclusion.....	31
Appendix A: File List.....	31
Appendix B: Testbench and Script Listings	31
B.1 Testbench (APB_Top_tb.v)	31
References	37

1. Introduction

1.1 Project Overview

This project implements a small AMBA APB (Advanced Peripheral Bus) subsystem in Verilog. A single APB master controller converts a simple system-side request/response interface into APB transfers. An address decoder selects one of two APB slave controllers, each of which contains its own 4096 x 32-bit memory. The slave responses (PREADY, PSLVERR, PRDATA) are multiplexed back to the master.

The design targets the APB4 feature set: it supports byte strobes (PSTRB) and protection attributes (PPROT) in addition to the APB3 wait-state (PREADY) and error (PSLVERR) signals. A self-contained testbench and a ModelSim/Questa run script are provided to compile, simulate and view the design.

1.2 Objectives

- Implement an APB master FSM (IDLE, SETUP, ACCESS) that accepts system requests and drives compliant APB transfers.
- Decode the address space into two slave regions and gate the slave select signals with the master select.
- Implement memory-backed slaves with byte-lane writes using PSTRB and out-of-range error reporting using PSLVERR.
- Return slave response signals to the master through a response multiplexer.
- Verify write, read, byte-strobe and consecutive transfers with a directed testbench.

1.3 Scope

In scope: the master controller, address decoder, slave controller (instantiated twice), top-level integration and response multiplexer, the testbench, and the simulation script.

Out of scope: bridges to other protocols (AXI/AHB), multiple masters or arbitration, slaves that insert wait states, use of PPROT for access control, low-power features, and formal verification.

1.4 Tools and Environment

Tool	Version	Purpose
Questa / ModelSim (vlog, vsim)		Compilation, simulation and waveform viewing through run.do
Xilinx Vivado		RTL elaboration and schematic generation
Icarus Verilog	12.0	Independent run of the testbench used to cross-check the console log in Section 7.3
Verilog / SystemVerilog		Design language (files are compiled with vlog -sv)

1.5 Definitions and Abbreviations

Term	Meaning
AMBA	Advanced Microcontroller Bus Architecture
APB	Advanced Peripheral Bus
PCLK / pclk	APB clock; all logic is clocked on its rising edge

Term	Meaning
PRESETn / prstn	Active-low asynchronous reset
PSEL	Slave select, driven by the master and decoded per slave
PENABLE	Marks the second and subsequent cycles (ACCESS phase) of a transfer
PREADY	Slave ready; low extends the ACCESS phase (wait state)
PSLVERR	Slave error response
PSTRB	Write byte-lane strobes (APB4)
PPROT	Protection attributes (APB4)
FSM	Finite state machine
DUT / TB	Device under test / testbench
RTL	Register transfer level

2. AMBA APB Protocol Background

2.1 Overview of AMBA APB

The AMBA APB protocol is a low-cost, low-power interface intended for peripheral registers and other low-bandwidth devices. It is non-pipelined: every transfer takes at least two clock cycles and only one transfer is in flight at a time. A single master (the APB bridge or requester) drives the address and control signals, and each slave has its own select line.

This project implements the APB4 revision of the protocol, which adds PSTRB and PPROT to the earlier APB3 definition (PREADY and PSLVERR).

2.2 Protocol Signals

Signal	Source	Description
PCLK	Clock source	Rising edge times all transfers.
PRESETn	System reset	Active-low reset.
PADDR	Master	Byte address of the transfer.
PPROT[2:0]	Master	Normal/privileged, secure/non-secure and data/instruction attributes.
PSELx	Master / decoder	One select per slave; asserted for the whole transfer.
PENABLE	Master	Low in the SETUP phase, high in the ACCESS phase.
PWRITE	Master	High for a write, low for a read.
PWDATA	Master	Write data.
PSTRB[3:0]	Master	One bit per byte lane of PWDATA; must be low for reads.
PREADY	Slave	Slave can complete the transfer; low inserts wait states.
PRDATA	Slave	Read data, valid when PREADY is high in the ACCESS phase.
PSLVERR	Slave	Error response, valid together with PREADY.

2.3 Transfer Phases

- IDLE: the default state. PSELx and PENABLE are low.
- SETUP: PSELx is asserted and the address, control and write data are driven. This phase lasts exactly one PCLK cycle.
- ACCESS: PENABLE is asserted while all other signals are held stable. The phase lasts until the slave drives PREADY high on a rising PCLK edge.

After the ACCESS phase the bus returns to IDLE, or directly to SETUP if another transfer is pending.

2.4 Write and Read Transfers

In a write, the master drives PWRITE, PADDR, PWDATA and PSTRB in the SETUP phase and holds them through the ACCESS phase; the slave captures the data at the end of the ACCESS phase when PREADY is high. In a read, PWRITE is low and the slave supplies PRDATA, which the master samples on the clock edge that ends the ACCESS phase with PREADY high.

If PREADY is low at the end of a cycle in the ACCESS phase, the transfer is extended by one wait state and all master outputs remain unchanged. PSLVERR is only meaningful in the last cycle of the transfer, when PSEL, PENABLE and PREADY are all high.

2.5 Byte Strokes and Protection (APB4)

PSTRB[n] enables the write of byte lane n (PWDATA[8n+7:8n]). A write with PSTRB = 4'b0011 therefore updates only the two least significant bytes. For read transfers all PSTRB bits must be driven low; this design forces PSTRB to 4'b0000 on reads. PPROT carries protection attributes from the master to the slave; the slaves in this design accept but do not use it.

3. Requirements and Specifications

3.1 Functional Requirements

ID	Requirement	Priority	Verified By
FR-01	The master shall accept a system request when req_valid and req_ready are both high, and perform an APB transfer consisting of a SETUP phase followed by an ACCESS phase.	High	Tests 1-4
FR-02	The master shall support both write and read transfers selected by req_write.	High	Tests 1, 2, 4
FR-03	The master shall drive PSTRB from req_strb on writes and 4'b0000 on reads.	High	Test 2; inspection
FR-04	The decoder shall select Slave 1 for 0x0000_0000-0x0000_3FFF and Slave 2 for 0x0000_4000-0x0000_7FFF.	High	Tests 1, 2, 4
FR-05	At most one slave select shall be asserted, and each select shall be qualified by the master PSEL.	High	Inspection
FR-06	Each slave shall provide a 4096 x 32-bit memory with byte-lane writes controlled by PSTRB.	High	Test 2
FR-07	Each slave shall complete transfers with zero wait states (PREADY = 1).	Medium	All tests; inspection
FR-08	A slave shall assert PSLVERR in the ACCESS phase when the word index of the access is not below MEMDEPTH.	Medium	Test 3; inspection

ID	Requirement	Priority	Verified By
FR-09	The master shall report completion with one-cycle done and <code>rsp_valid</code> pulses, with <code>rsp_rdata</code> and <code>rsp_error</code> captured from the bus.	High	Tests 1-4
FR-10	The master shall remain in ACCESS while PREADY is low and hold all bus outputs.	High	Inspection
FR-11	The master shall move directly from ACCESS to SETUP when a new request is pending as a transfer completes.	Medium	Inspection
FR-12	An asynchronous active-low reset shall return the master to IDLE with <code>req_ready</code> = 1.	High	Reset at start of testbench
FR-13	When no slave is selected the response multiplexer shall return PREADY = 1, PSLVERR = 0 and PRDATA = 0.	Medium	Inspection

3.2 Design Parameters

Parameter	Value	Notes
DWIDTH	32	Data width of the request, response and APB data buses.
ADDRWIDTH	32	Address width.
MEMDEPTH	4096	Words per slave memory (16 KiB per slave).
BASEADD (Slave 1)	32'h0000_0000	Subtracted from PADDR to form the local address.
BASEADD (Slave 2)	32'h0000_4000	Subtracted from PADDR to form the local address.
Clock period (testbench)	10 ns	100 MHz; timescale 1 ns / 1 ps.

3.3 Assumptions and Constraints

- Single master and a single clock domain (`pclk`).
- Accesses are word-aligned; the two least significant address bits are ignored by the slaves.
- The slave memories are not initialised or reset, so locations that have not been written contain unknown values in simulation.
- The system side raises `req_valid` only while `req_ready` is high (the provided testbench holds `req_valid` for exactly one cycle).

4. System Architecture

4.1 Top-Level Block Diagram

The top module `apb_subsystem_top` instantiates the master (`u_apb_master`), the decoder (`u_apb_decoder`) and two slave controllers (`u_slave_1`, `u_slave_2`). Two AND gates qualify the decoder outputs with the master PSEL, and a set of multiplexers returns the selected slave response to the master. The elaborated RTL schematic is shown below.

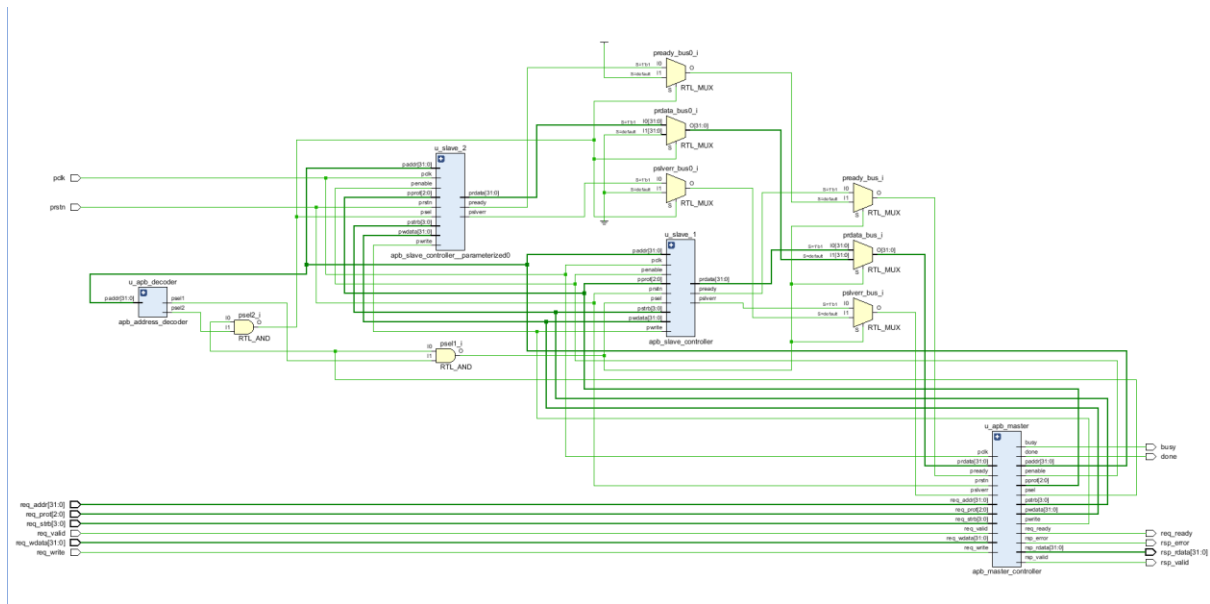


Figure 4.1 - Top-level RTL schematic of apb_subsystem_top

4.2 Module Hierarchy

Instance	Module	Description
dut	apb_subsystem_top	Top level instantiated by the testbench tb_apb_subsystem.
u_apb_master	apb_master_controller	System-to-APB master FSM and request/response registers.
u_apb_decoder	apb_address_decoder	Combinational decode of PADDR into psel1 / psel2.
u_slave_1	apb_slave_controller	Slave with BASEADD = 0x0000_0000 and a 4096 x 32 memory.
u_slave_2	apb_slave_controller	Slave with BASEADD = 0x0000_4000 and a 4096 x 32 memory.
(assign statements)	in apb_subsystem_top	psel1/psel2 gating and the pready / pslverr / prdata response multiplexers.

4.3 Data and Control Flow

1. The testbench (system side) presents req_valid, req_write, req_addr, req_wdata, req_strb and req_prot while req_ready is high.
2. On the next clock edge the master latches the request into its APB output registers, asserts psel and drops req_ready; busy goes high.
3. The decoder maps paddr to psel1_raw or psel2_raw; each is ANDed with the master psel to form psel1 / psel2.
4. In the ACCESS phase (penable high) the selected slave performs the memory write or read and returns pready, pslverr and prdata.
5. The response multiplexer forwards the selected slave's signals to the master.
6. When pready is high the master captures prdata and pslverr into rsp_rdata and rsp_error, pulses done and rsp_valid, and either returns to IDLE (req_ready = 1, busy = 0) or starts the next request.

4.4 Memory Map

Slave	Base Address	End Address	Size	Description
Slave 1	0x0000_0000	0x0000_3FFF	16 KiB	4096 x 32-bit words, selected by psel1.
Slave 2	0x0000_4000	0x0000_7FFF	16 KiB	4096 x 32-bit words, selected by psel2.
Unmapped	0x0000_8000	0xFFFF_FFFF	-	No select asserted; the response multiplexer returns its default values (see Section 5.4).

4.5 Top-Level Interface

Clock, Reset and System Request Interface

Signal	Direction	Width	Description
pclk	Input	1	APB clock.
prstn	Input	1	Active-low asynchronous reset.
req_valid	Input	1	Request valid. Accepted when req_ready is also high.
req_write	Input	1	1 = write, 0 = read.
req_addr	Input	ADDRWIDTH	Request byte address.
req_wdata	Input	DWIDTH	Write data.
req_strb	Input	4	Write byte strobes.
req_prot	Input	3	Protection attributes forwarded to PPROT.

System Response Interface

Signal	Direction	Width	Description
req_ready	Output	1	High when the master can accept a new request.
rsp_valid	Output	1	One-cycle pulse when a response is available.
rsp_error	Output	1	PSLVERR captured for the last transfer.
busy	Output	1	High while a transfer is in progress.
done	Output	1	One-cycle pulse when a transfer completes.
rsp_rdata	Output	DWIDTH	Read data captured for the last transfer.

5. Detailed Design

5.1 APB Master Controller

Purpose

apb_master_controller converts the system request/response handshake into APB transfers. It uses a three-state FSM to sequence the SETUP and ACCESS phases, registers the request onto the APB outputs and captures the slave response.

Parameters

Parameter	Default	Description
DWIDTH	32	Width of data buses.
ADDRWIDTH	32	Width of address buses.

Port List

Signal	Direction	Width	Description
pclk	Input	1	Clock; all registers use the rising edge.
prstn	Input	1	Active-low asynchronous reset.
req_valid	Input	1	System request valid.
req_write	Input	1	System request direction.
req_addr	Input	ADDRWIDTH	System request address.
req_wdata	Input	DWIDTH	System write data.
req_strb	Input	4	System write byte strobes.
req_prot	Input	3	System protection attributes.
pready	Input	1	Muxed slave ready.
pslverr	Input	1	Muxed slave error.
prdata	Input	DWIDTH	Muxed slave read data.
req_ready	Output (reg)	1	Master can accept a request. Reset value 1.
rsp_valid	Output (reg)	1	Response valid pulse.
rsp_error	Output (reg)	1	Captured pslverr; holds until the next response.
busy	Output (reg)	1	Transfer in progress.
done	Output (reg)	1	Transfer complete pulse.
rsp_rdata	Output (reg)	DWIDTH	Captured prdata; holds until the next response.

Signal	Direction	Width	Description
paddr	Output (reg)	ADDRWIDTH	APB address.
pprot	Output (reg)	3	APB protection attributes.
psel	Output (reg)	1	Master select, high during SETUP and ACCESS.
penable	Output (wire)	1	Continuous assignment: high when the FSM is in ACCESS.
pwrite	Output (reg)	1	APB write/read control.
pwdata	Output (reg)	DWIDTH	APB write data.
pstrb	Output (reg)	4	APB byte strobes (0000 for reads).

FSM Description

State	Encoding	Description
IDLE	2'b00	No transfer in progress. psel = 0. A request is accepted here when req_valid and req_ready are high.
SETUP	2'b01	One cycle with psel = 1 and penable = 0. Address and control are stable on the bus.
ACCESS	2'b10	psel = 1 and penable = 1. Remains until pready is high.
(unused)	2'b11	Not reachable; the default branch returns to IDLE.

State Transition Table

Current State	Condition	Next State	Outputs / Actions
IDLE	req_valid = 0	IDLE	psel = 0, req_ready = 1, busy = 0.
IDLE	req_valid = 1	SETUP	Latch paddr, pwdata, pwrite, pprot; pstrb = req_write ? req_strb : 0; psel = 1; req_ready = 0; busy = 1.
SETUP	always	ACCESS	Outputs held; penable becomes 1 in the next state.
ACCESS	pready = 0	ACCESS	Wait state: all outputs held.
ACCESS	pready = 1, req_valid = 0	IDLE	rsp_rdata = prdata; rsp_error = pslverr; done = 1; rsp_valid = 1; psel = 0; busy = 0; req_ready = 1.
ACCESS	pready = 1, req_valid = 1	SETUP	Response captured as above; new request latched; psel stays 1; busy = 1; req_ready = 0.

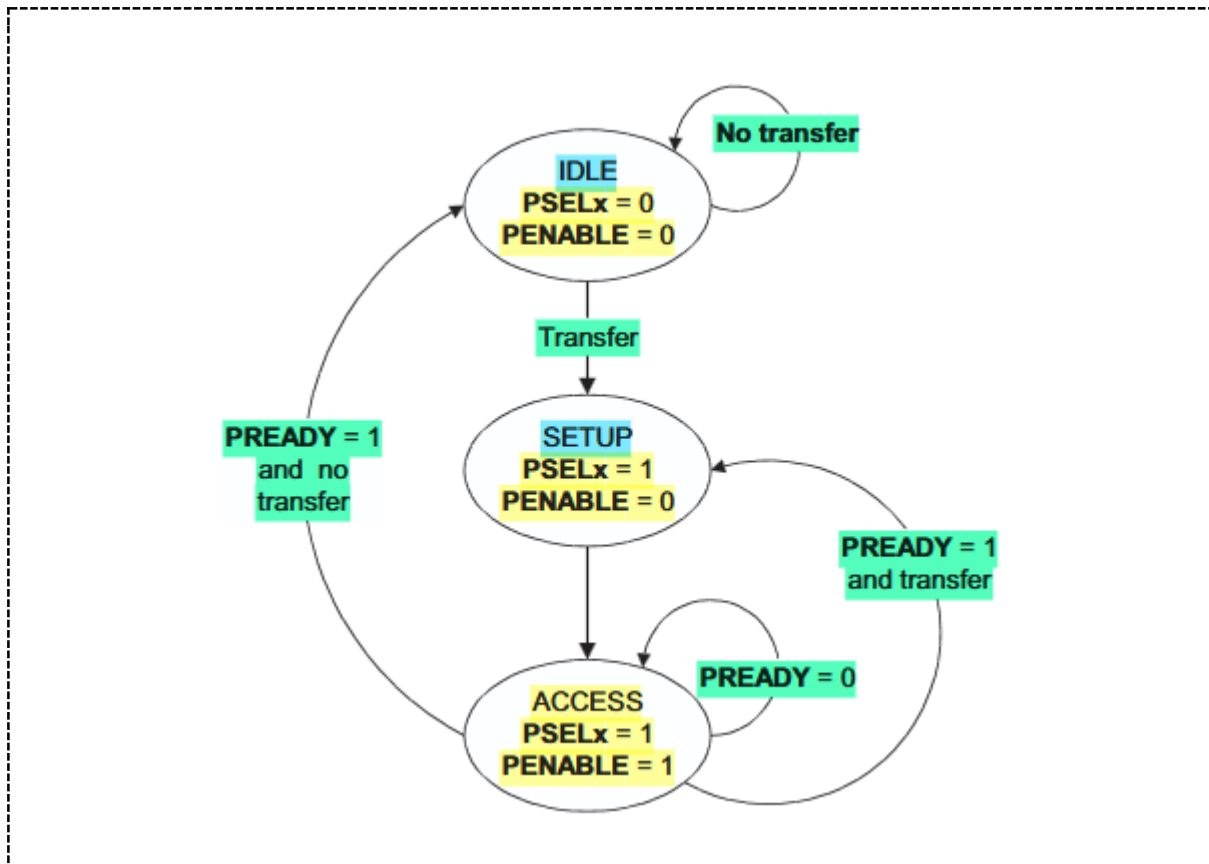


Figure 5.1 - Master controller FSM

Single-Transfer Timing

The following sequence was observed in simulation for one transfer (cycle numbers are PCLK cycles, values are those present before the rising edge that ends each cycle):

Cycle	State	psel	penable	req_ready	busy	done / rsp_valid	Comment
0	IDLE	0	0	1	0	0	req_valid = 1 is sampled; request accepted.
1	SETUP	1	0	0	1	0	Address and control driven.
2	ACCESS	1	1	0	1	0	Slave responds with pready = 1.
3	IDLE	0	0	1	0	1	Response registered; done and rsp_valid pulse.

Functional Description

The FSM has two processes: a sequential process that updates current_state on the rising edge of pclk (or resets it asynchronously), and a combinational process that computes next_state. A separate clocked process drives all registered outputs from current_state. penable is a continuous assignment equal to (current_state == ACCESS).

Because the outputs are registered and decided from current_state, psel and the latched address/control appear on the bus at the same edge at which the FSM enters SETUP. The response

(rsp_rdata, rsp_error, done, rsp_valid) is registered at the edge that ends the ACCESS phase and is therefore visible in the following cycle.

Output Register Behavior

- done and rsp_valid default to 0 every cycle and are set only in the cycle after a transfer completes, so they are single-cycle pulses.
- req_ready is 1 in IDLE unless a request is being accepted, is cleared when a request is accepted, and is set again when a transfer finishes with no request pending.
- busy is 1 from the acceptance of a request until a transfer finishes with no pending request.
- rsp_rdata and rsp_error are only updated when pready is high in ACCESS and otherwise hold their last value.
- pstrb is forced to 4'b0000 for reads.

Back-to-Back Transfer Handling

When pready is high in ACCESS and req_valid is also high, the master latches the new request in the same edge, keeps psel high and moves directly to SETUP, so consecutive transfers need no IDLE cycle. Because req_ready is low while a transfer is running, this path is only used if the system side keeps req_valid asserted; the provided testbench pulses req_valid for one cycle and therefore does not exercise it (see Section 9.2).

Source Code

```
module apb_master_controller #(
    parameter DWIDTH = 32,
    parameter ADDRWIDTH = 32
)(
    // clock and reset
    input pclk, prstn,

    // system request interface
    input req_valid, req_write,
    input [ADDRWIDTH - 1 : 0] req_addr,
    input [DWIDTH - 1 : 0] req_wdata,
    input [3:0] req_strb,
    input [2:0] req_prot,

    // APB Bus Interface (inputs from slaves)
    input pready, pslverr,
    input [DWIDTH - 1 : 0] prdata,

    // system response interface
    output reg req_ready, rsp_valid, rsp_error, busy, done,
    output reg [DWIDTH - 1 : 0] rsp_rdata,

    // APB Bus Interface (outputs to slaves)
    output reg [ADDRWIDTH - 1 : 0] paddr,
    output reg [2:0] pprot,
    output reg psel,
    output wire penable,
```

```

output reg pwrite,
output reg [DWIDTH - 1 : 0] pwdata,
output reg [3:0] pstrb
);
// FSM State Encoding
localparam IDLE = 2'b00,
            SETUP = 2'b01,
            ACCESS = 2'b10;

reg [1:0] current_state, next_state;

// Asserts continuously during the ACCESS state
assign penable = (current_state == ACCESS);

// current state logic
always @(posedge pclk or negedge prstn) begin
    if (!prstn) begin
        current_state <= IDLE;
    end else begin
        current_state <= next_state;
    end
end

// next state logic
always @(*) begin
    case (current_state)
        IDLE: begin
            if (!req_valid)
                next_state = IDLE;
            else
                next_state = SETUP;
        end

        SETUP: begin
            next_state = ACCESS;
        end

        ACCESS: begin
            if (!pready)
                next_state = ACCESS;
            else begin
                if (req_valid)
                    next_state = SETUP;
                else
                    next_state = IDLE;
            end
        end
    endcase
end

```

```

        end

        default: next_state = IDLE;
    endcase
end

// output logic
always @(posedge pclk or negedge prstn) begin

    if (!prstn) begin
        // default system response signals
        req_ready <= 1;
        rsp_valid <= 0;
        rsp_error <= 0;
        busy <= 0;
        done <= 0;
        rsp_rdata <= 0;

        // default APB bus signals
        paddr <= 0;
        pprot <= 0;
        psel <= 0;
        pwrite <= 0;
        pwrite <= 0;
        pstrb <= 0;

    end else begin
        done <= 0;
        rsp_valid <= 0;

        case (current_state)

            IDLE: begin
                psel <= 0;

                if (req_valid && req_ready) begin
                    paddr <= req_addr;
                    pwrite <= req_wdata;
                    pwrite <= req_write;
                    pstrb <= req_write ? req_strb : 4'b0000;
                    pprot <= req_prot;
                    psel <= 1;
                    req_ready <= 0;
                    busy <= 1;
                end else begin
                    req_ready <= 1;

```

```

        busy <= 0;
    end
end

ACCESS: begin
    if (pready) begin
        // capture slave response
        rsp_rdata <= prdata;
        rsp_error <= pslverr;
        done <= 1;
        rsp_valid <= 1;

        if (req_valid) begin
            paddr <= req_addr;
            pwidth <= req_wdata;
            pwrite <= req_write;
            pstrb <= req_write ? req_strb :
4'b0000;

            pprot <= req_prot;
            psel <= 1;
            busy <= 1;
            req_ready <= 0;
        end else begin
            // Return to IDLE
            busy <= 1'b0;
            req_ready <= 1'b1;
            psel <= 1'b0;
        end
    end
end
endcase
end
end

endmodule

```

5.2 APB Address Decoder

Purpose

apb_address_decoder is a purely combinational block that converts the APB address into one select line per slave.

Parameters

Parameter	Default	Description
ADDRWIDTH	32	Width of the address input.

Port List

Signal	Direction	Width	Description
paddr	Input	ADDRWIDTH	APB address from the master.
psel1	Output (reg)	1	High when the address is in the Slave 1 range.
psel2	Output (reg)	1	High when the address is in the Slave 2 range.

Address Ranges

Select Output	Address Range	Target Slave
psel1	0x0000_0000 - 0x0000_3FFF	u_slave_1
psel2	0x0000_4000 - 0x0000_7FFF	u_slave_2
(none)	Everything else	No slave selected

Functional Description

An always @(*) block first clears both selects and then sets at most one of them using an if / else-if chain, so the outputs are mutually exclusive and glitch-free defaults are guaranteed. Addresses above 0x0000_7FFF leave both selects low. The decoder does not look at psel; the qualification with the master select is done in the top module.

Block Diagram / Waveform

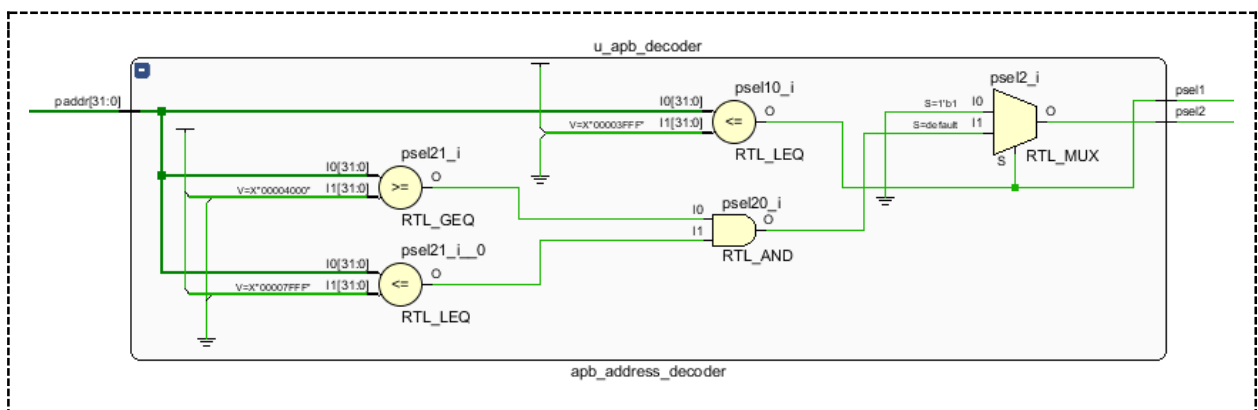


Figure 5.2 - Address decoder

Source Code

```

module apb_address_decoder #(
    parameter ADDRWIDTH = 32
)(
    input wire [ADDRWIDTH-1:0] paddr,
    output reg psel1, // Select for Slave 1 (0x0000_0000 -
0x0000_3FFF)
    output reg psel2 // Select for Slave 2 (0x0000_4000 -
0x0000_7FFF)
);

    always @(*) begin
        psel1 = 1'b0;
        psel2 = 1'b0;

        if (paddr >= 32'h0000_0000 && paddr <= 32'h0000_3FFF)
begin
            psel1 = 1'b1;
        end else if (paddr >= 32'h0000_4000 && paddr <=
32'h0000_7FFF) begin
            psel2 = 1'b1;
        end
    end
endmodule

```

5.3 APB Slave Controller

Purpose

apb_slave_controller is an APB slave containing a synchronous memory. It is instantiated twice with different base addresses to create the two regions of the memory map.

Parameters

Parameter	Default	Description
DWIDTH	32	Data width.
ADDRWIDTH	32	Address width.
BASEADD	32'h0000_0000	Base address of the region; subtracted from paddr.
MEMDEPTH	4096	Number of 32-bit words in the memory.

Port List

Signal	Direction	Width	Description
pclk	Input	1	Clock.
prstn	Input	1	Reset input (present but not used inside the module).
paddr	Input	ADDRWIDTH	APB address.
pprot	Input	3	Protection attributes (not used).
psel	Input	1	Slave select from the top-level gating.
penable	Input	1	ACCESS-phase indicator.
pwrite	Input	1	Write / read control.
pwwdata	Input	DWIDTH	Write data.
pstrb	Input	4	Write byte strobes.
pready	Output	1	Constant 1 (zero wait states).
pslverr	Output	1	High in ACCESS when the word index is not below MEMDEPTH.
prdata	Output (reg)	DWIDTH	Read data, registered on the clock edge of a read access.

Internal Memory

The slave holds a reg array mem[0:MEMDEPTH-1] of DWIDTH-bit words. The local address is paddr - BASEADD and the word index is local_addr >> 2, so byte addresses that differ only in bits [1:0] refer to the same word. The array is indexed with word_index[11:0], which matches the default depth of 4096 words.

Byte-Strobe Write Behavior

When psel, penable and pwrite are all high, each strobe bit enables the write of one byte lane: pstrb[0] writes bits [7:0], pstrb[1] bits [15:8], pstrb[2] bits [23:16] and pstrb[3] bits [31:24]. Lanes whose strobe is low keep their previous value, which allows partial-word updates such as the pstrb = 4'b0011 case used in Test 2.

Error Generation

pslverr = (psel && penable) ? (word_index >= MEMDEPTH) : 0. The error is therefore only driven during the ACCESS phase and is derived combinatorially from the address. With the default memory map each slave window (16 KiB) is exactly MEMDEPTH words, so this condition can only become true if MEMDEPTH is reduced or the window is enlarged (see Section 9.2).

Wait States

pready is tied to 1, so every access completes in the first ACCESS cycle with no wait states.

Read Path Timing

On a read (psel, penable high, pwrite low) prdata is assigned inside the clocked always block, so it is a register that updates at the clock edge that ends the ACCESS cycle. Since the master samples prdata on that same edge, it sees the value the register held before the edge, which is the result of the previous read (see Section 9.2).

Block Diagram / Waveform

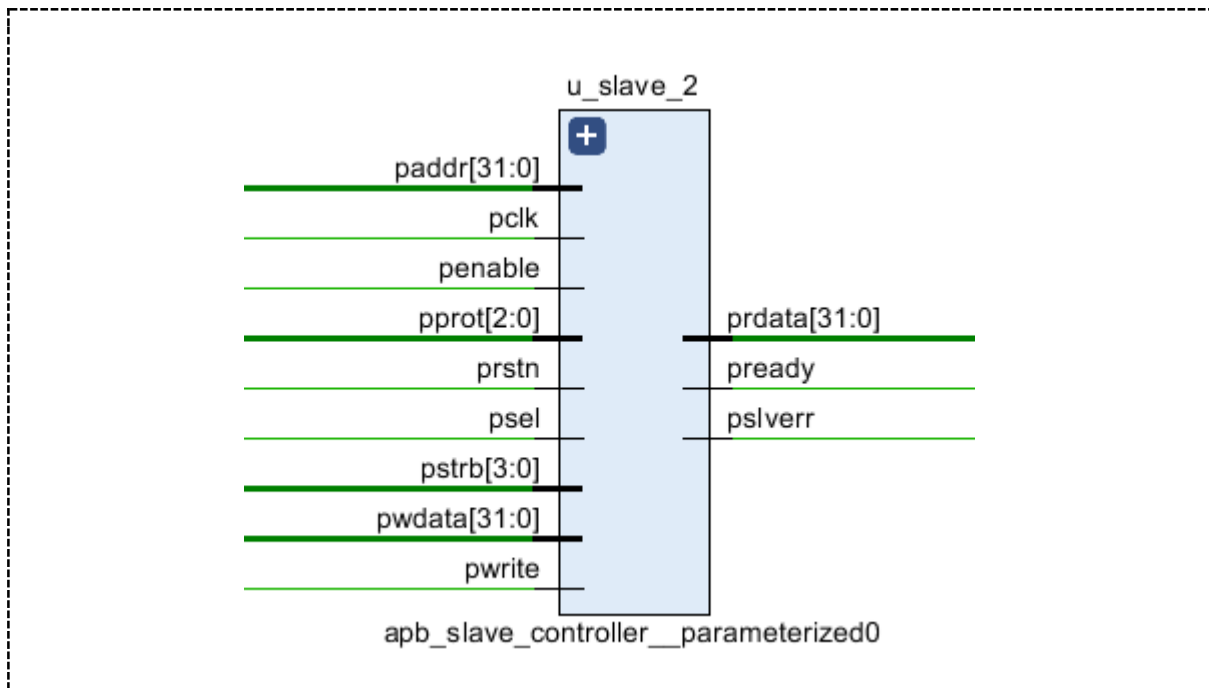


Figure 5.3 - Slave controller access

Source Code

```

module apb_slave_controller #(
    parameter DWIDTH = 32,
    parameter ADDRWIDTH = 32,
    parameter BASEADD = 32'h0000_0000,
    parameter MEMDEPTH = 4096
)()
    // clock and reset
    input pclk, prstn,

    // APB Bus Interface (Inputs from master)
    input [ADDRWIDTH - 1 : 0] paddr,
    input [2:0] pprot,
    input psel, penable, pwrite,
    input [DWIDTH - 1 : 0] pwrite,
    input [3:0] pstrb,

    //APB Bus Interface (Outputs to Master)
    output pready, pslverr,
    output reg [DWIDTH - 1 : 0] prdata
);

    // Built-in 4096x32-bit SRAM array inside the slave wrapper
    reg [DWIDTH - 1 : 0] mem [0 : MEMDEPTH - 1];

```

```

// Local address offset (removes the base address)
wire [31:0] local_addr = paddr - BASEADD;

// Shift right by 2 to map 4-byte aligned addresses
wire [ADDRWIDTH-1:0] word_index = local_addr >> 2;

// Generate error if master attempts an out-of-bounds address
access
assign pslverr = (psel && penable) ? (word_index >= MEMDEPTH)
: 1'b0;

// 0 wait cycles
assign pready = 1'b1;

always @(posedge pclk) begin
    if (psel && penable) begin
        if (pwrite) begin
            if (pstrb[0]) mem[word_index[11:0]][ 7: 0] <=
pdata[ 7: 0];
            if (pstrb[1]) mem[word_index[11:0]][15: 8] <=
pdata[15: 8];
            if (pstrb[2]) mem[word_index[11:0]][23:16] <=
pdata[23:16];
            if (pstrb[3]) mem[word_index[11:0]][31:24] <=
pdata[31:24];
        end else begin
            prdata <= mem[word_index[11:0]];
        end
    end
end
endmodule

```

5.4 Top Level: Select Gating and Response Multiplexer

Purpose

apb_subsystem_top wires the blocks together, qualifies the decoder outputs with the master select, and multiplexes the slave responses back to the master.

Select Gating

Signal	Expression	Meaning
psel1	psel_master & psel1_raw	Select for Slave 1, only during a transfer.
psel2	psel_master & psel2_raw	Select for Slave 2, only during a transfer.

Response Multiplexer

Signal	Expression
pready_bus	psel1 ? pready_s1 : (psel2 ? pready_s2 : 1'b1)
pslverr_bus	psel1 ? pslverr_s1 : (psel2 ? pslverr_s2 : 1'b0)
prdata_bus	psel1 ? prdata_s1 : (psel2 ? prdata_s2 : {DWIDTH{1'b0}})

Default Behavior When No Slave Is Selected

If neither select is high the multiplexer returns pready = 1, pslverr = 0 and prdata = 0. This applies to the idle cycles between transfers and also to any access to an unmapped address (above 0x0000_7FFF): such a transfer completes normally after the ACCESS phase, without an error and with read data of zero.

Parameter Propagation

DWIDTH, ADDRWIDTH and MEMDEPTH are parameters of the top module and are passed to the master, decoder and slaves. The slave base addresses are hard-coded in the instantiations (0x0000_0000 and 0x0000_4000).

Source Code

```
module apb_subsystem_top #(
    parameter DWIDTH      = 32,
    parameter ADDRWIDTH   = 32,
    parameter MEMDEPTH    = 4096
)(
    // Clock and Reset
    input  pclk,
    input  prstn,

    // System Request Interface (to APB Master)
    input  req_valid,
    input  req_write,
    input  [ADDRWIDTH-1:0] req_addr,
    input  [DWIDTH-1:0] req_wdata,
    input  [3:0] req_strb,
    input  [2:0] req_prot,

    // System Response Interface (from APB Master)
    output req_ready,
    output rsp_valid,
    output rsp_error,
    output busy,
    output done,
    output [DWIDTH-1:0] rsp_rdata
);
```

```

// Internal Interconnect Wires

// Master Output / APB Bus Signals
wire [ADDRWIDTH-1:0] paddr;
wire [2:0] pprot;
wire psel_master; // Base psel output from Master
wire penable;
wire pwrite;
wire [DWIDTH-1:0] pwwdata;
wire [3:0] pstrb;

// Decoder Output Signals
wire psel1_raw;
wire psel2_raw;

// Qualified Slave Selects (Gated with Master's psel)
wire psel1 = psel_master & psel1_raw;
wire psel2 = psel_master & psel2_raw;

// Slave Outputs to Mux
wire pready_s1, pready_s2;
wire pslverr_s1, pslverr_s2;
wire [DWIDTH-1:0] prdata_s1, prdata_s2;

// Multiplexed Bus Feedback Signals (Selected based on which
Slave is active)
wire pready_bus;
wire pslverr_bus;
wire [DWIDTH-1:0] prdata_bus;

// APB Master Controller Instantiation
apb_master_controller #(
    .DWIDTH(DWIDTH),
    .ADDRWIDTH(ADDRWIDTH)
) u_apb_master (
    .pclk      (pclk),
    .prstn     (prstn),

    // System Request Interface
    .req_valid (req_valid),
    .req_write (req_write),
    .req_addr  (req_addr),
    .req_wdata (req_wdata),
    .req_strb  (req_strb),
    .req_prot  (req_prot),

```

```

    // System Response Interface
    .req_ready (req_ready),
    .rsp_valid (rsp_valid),
    .rsp_error (rsp_error),
    .busy      (busy),
    .done      (done),
    .rsp_rdata (rsp_rdata),

    // APB Bus Signals
    .pready    (pready_bus),
    .pslverr   (pslverr_bus),
    .prdata    (prdata_bus),
    .paddr     (paddr),
    .pprot     (pprot),
    .psel      (psel_master),
    .penable   (penable),
    .pwrite    (pwrite),
    .pwwdata   (pwwdata),
    .pstrb     (pstrb)
);

// APB Address Decoder Instantiation
apb_address_decoder #(
    .ADDRWIDTH(ADDRWIDTH)
) u_apb_decoder (
    .paddr    (paddr),
    .psel1    (psel1_raw),
    .psel2    (psel2_raw)
);

// APB Slave Controller 1 (Base Address: 0x0000_0000)
apb_slave_controller #(
    .DWIDTH    (DWIDTH),
    .ADDRWIDTH (ADDRWIDTH),
    .BASEADD    (32'h0000_0000),
    .MEMDEPTH   (MEMDEPTH)
) u_slave_1 (
    .pclk       (pclk),
    .prstn      (prstn),
    .paddr      (paddr),
    .pprot      (pprot),
    .psel       (psel1),
    .penable    (penable),

```

```

        .pwrite      (pwrite),
        .pwrite      (pwrite),
        .pwrite      (pwrite),
        .pready      (pready_s1),
        .pslverr     (pslverr_s1),
        .prdata      (prdata_s1)
    );

    // APB Slave Controller 2 (Base Address: 0x0000_4000)
    apb_slave_controller #(
        .DWIDTH      (DWIDTH),
        .ADDRWIDTH    (ADDRWIDTH),
        .BASEADD      (32'h0000_4000),
        .MEMDEPTH      (MEMDEPTH)
    ) u_slave_2 (
        .pclk          (pclk),
        .prstn         (prstn),
        .paddr         (paddr),
        .pprot         (pprot),
        .psel          (psel2),
        .penable       (penable),
        .pwrite        (pwrite),
        .pwrite        (pwrite),
        .pwrite        (pwrite),
        .pready        (pready_s2),
        .pslverr       (pslverr_s2),
        .prdata        (prdata_s2)
    );

    // APB Response Multiplexer
    // Route slave outputs back to the master based on active
select signals
    assign pready_bus  = psel1 ? pready_s1   : (psel2 ?
pready_s2   : 1'b1);
    assign pslverr_bus = psel1 ? pslverr_s1  : (psel2 ? pslverr_s2
: 1'b0);
    assign prdata_bus  = psel1 ? prdata_s1   : (psel2 ?
prdata_s2   : {DWIDTH{1'b0}});

endmodule

```

6. Verification

6.1 Verification Strategy

The design is verified with a directed, self-contained testbench (tb_apb_subsystem) that drives the system request interface through two helper tasks and reports each transfer with \$display. Results are inspected in the console output and in the waveform groups created by run.do. The testbench does not contain a scoreboard or automatic checkers other than one data comparison in Test 2 and a check of rsp_error in Test 3; functional coverage was not collected.

6.2 Testbench Architecture

- The DUT is apb_subsystem_top with DWIDTH = 32, ADDRWIDTH = 32 and MEMDEPTH = 4096.
- Clock: pclk toggles every 5 ns (10 ns period, 100 MHz).
- Reset: prstn is held low for 20 ns and then released, followed by a 10 ns pause before the first test.
- apb_write(addr, data, strb): waits for req_ready, drives req_valid, req_write = 1 and the request fields for one clock cycle, then waits for done and prints [WRITE SUCCESS] or [WRITE ERROR] depending on rsp_error.
- apb_read(addr, data): same handshake with req_write = 0 and req_strb = 0; returns rsp_rdata and prints [READ SUCCESS] or [READ ERROR].
- read_data_buf stores the value returned by apb_read for comparison.
- Each transaction takes five PCLK cycles (50 ns) from one request to the next, with the bus idle for the remaining cycles.

6.3 Test Plan

Test ID	Test Name	Description	Expected Result	Status
T1	Full-word access (Slave 1)	Write 0xDEADBEEF to 0x0 and 0xCAFEBAFE to 0x4 with strobe 1111, then read both back.	Reads return the written words with no error.	Writes OK; read data wrong (see 6.4, 9.2)
T2	Byte-strobe masking (Slave 2)	Write 0 to 0x4000, write 0xFFFF1234 with strobe 0011, read back.	Read returns 0x00001234.	Read data wrong (see 6.4, 9.2)
T3	Out-of-bounds PSLVERR	Write 0xA5A5A5A5 to 0x4000 (0x3FFF + 1) and check rsp_error.	PSLVERR asserted.	Not triggered (see 6.4, 9.2)
T4	Back-to-back transfers	Write 0x4008 and 0x400C, then read both.	Reads return 0x11112222 and 0x33334444.	Read data wrong (see 6.4, 9.2)

6.4 Test Case Details

Test 1: Full-Word Access

Stimulus: apb_write(0x0000_0000, 0xDEAD_BEEF, 4'b1111); apb_write(0x0000_0004, 0xCAFE_BABE, 4'b1111); apb_read(0x0000_0000); apb_read(0x0000_0004).

Expected: Both writes complete without error; the reads return 0xDEADBEEF and 0xCAFEBAFE.

Observed: Both writes complete with [WRITE SUCCESS]. The first read returns an unknown value (0xxxxxxx) and the second read returns 0xDEADBEEF, the content of the address read before it. The message "TEST 1 PASSED" is printed unconditionally because the test contains no comparison.

Test 2: Byte-Strobe Masking

Stimulus: `apb_write(0x4000, 0x0000_0000, 4'b1111); apb_write(0x4000, 0xFFFF_1234, 4'b0011); apb_read(0x4000).`

Expected: The upper two bytes stay 0x0000, so the read returns 0x0000_1234 and "Byte Strobe Verification PASSED" is printed.

Observed: Both writes complete. The read returns an unknown value, the comparison with 0x0000_1234 is false, and no PASSED message is printed. The strobed write itself is accepted by the slave; its effect becomes visible in the next read of Slave 2 (0x00001234 is returned by the first read in Test 4).

Test 3: Out-of-Bounds Error Detection

Stimulus: `apb_write(0x0000_3FFF + 1, 0xA5A5_A5A5, 4'b1111),` i.e. address 0x0000_4000.

Expected (testbench intent): PSLVERR is asserted for an address one word past the end of the memory.

Observed: The decoder maps 0x4000 to Slave 2 (local offset 0, word 0), which is a valid location, so the write completes with [WRITE SUCCESS], `rsp_error` stays low and the "PSLVERR correctly generated" message is not printed. The write overwrites word 0 of Slave 2.

Test 4: Back-to-Back Transfers

Stimulus: `apb_write(0x4008, 0x1111_2222); apb_write(0x400C, 0x3333_4444); apb_read(0x4008); apb_read(0x400C).`

Expected: Reads return 0x11112222 and 0x33334444.

Observed: Both writes complete. The reads return 0x00001234 and 0x11112222; each read returns the data of the previous read from Slave 2. "Back-To-Back Stream PASSED" is printed unconditionally. The transfers are issued one at a time through IDLE, so the ACCESS-to-SETUP path is not exercised.

6.5 Coverage

No simulator coverage was collected. The table lists the features of the design against the tests that exercise them.

Feature	Exercised By	Covered
Write to Slave 1	T1	Yes
Read from Slave 1	T1	Yes (data incorrect, see 9.2)
Write to Slave 2	T2, T3, T4	Yes
Read from Slave 2	T2, T4	Yes (data incorrect, see 9.2)
Full-word strobe (1111)	T1, T2, T3, T4	Yes
Partial strobe (0011)	T2	Yes, one pattern only

Feature	Exercised By	Covered
Other strobe patterns (0001, 0010, 0100, 1000, ...)	-	No
PSLVERR from a slave	T3 (intended)	No
Wait states (pready low)	-	No (slaves have zero wait states)
Access to an unmapped address	-	No
ACCESS-to-SETUP back-to-back path	-	No
Reset during a transfer	-	No (reset only at start)
Non-zero req_prot	-	No

7. Simulation

7.1 How to Run

1. Place APB_Address_Decoder.v, APB_Master_Controller.v, APB_Slave_Controller.v, APB_Top_Module.v, APB_Top_tb.v and run.do in one folder.
2. Start Questa / ModelSim and change the current directory to that folder.
3. Run "do run.do" in the transcript window.

The script performs the following steps:

- Deletes any existing work library, creates a new one (vlib) and maps it (vmap).
- Compiles the five source files with vlog -sv in dependency order: decoder, master, slave, top, testbench.
- Starts the simulation of work.tb_apb_subsystem with full visibility (vsim -voptargs="+acc").
- Defines the APB_STATE_T radix so the master state prints as IDLE, SETUP or ACCESS.
- Adds the signals to the wave window in ten colour-coded groups (table below) and configures the wave display.
- Runs the simulation to completion (run -all) and zooms the waveform to full range.

Wave Group	Contents	Colour
Clock_and_Reset	pclk, prstn	Yellow
System_Request_Interface	req_valid, req_write, req_addr, req_wdata, req_strb, req_prot	Green
System_Response_Interface	req_ready, rsp_valid, rsp_error, busy, done, rsp_rdata	Sky Blue
APB_Master_Bus_Outputs	psel_master, penable, pwrite, paddr, pwdata, pstrb, pprot	Orange
Address_Decoder	psel1_raw, psel2_raw, psel1, psel2	Cyan
Slave_1_Interface	psel, penable, pready_s1, pslverr_s1, prdata_s1, local_addr, word_index	Light Blue
Slave_2_Interface	psel, penable, pready_s2, pslverr_s2, prdata_s2, local_addr, word_index	Coral
Muxed_Bus_Response	pready_bus, pslverr_bus, prdata_bus	Red

Wave Group	Contents	Colour
Master_Internal_FSM	current_state, next_state (APB_STATE_T radix)	Magenta
TB_Variables	read_data_buf	White

7.2 Waveform Results

Suggested captures: the full simulation, one write transfer, one read transfer, the byte-strobe write, and the unmapped-address decode of Test 3. Each figure should show the wave groups listed above.

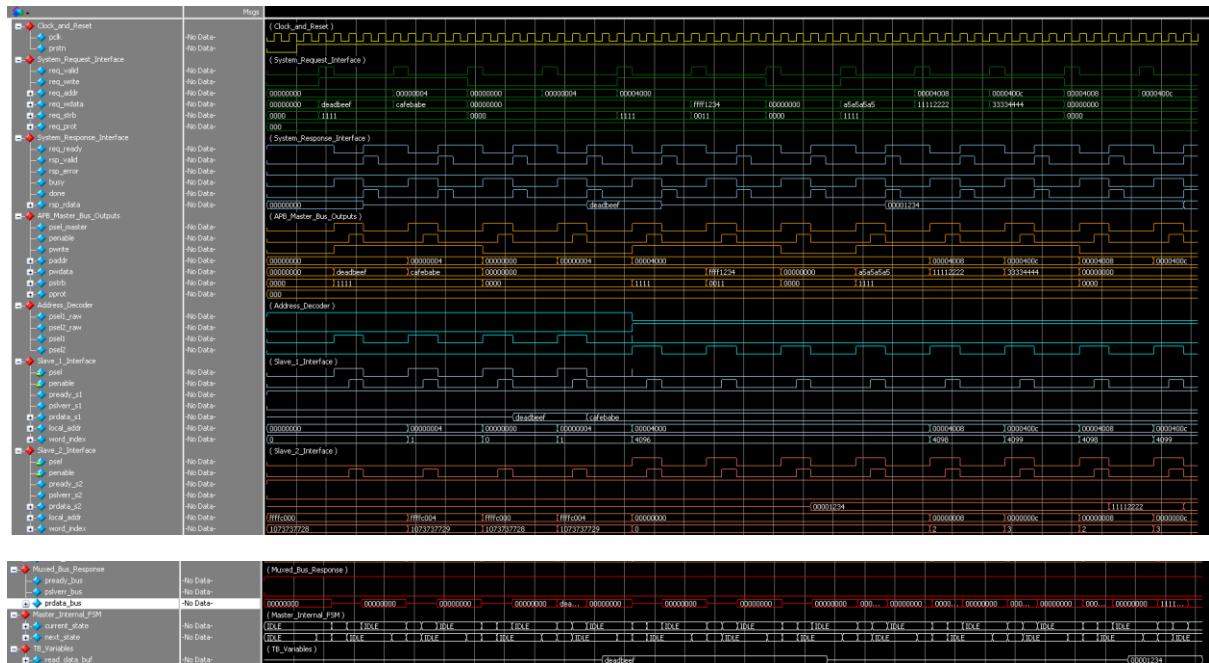


Figure 7.1 - Complete simulation

7.3 Console Log Output

The transcript below was produced by running the supplied testbench (Icarus Verilog 12.0). The \$display output is generated by the testbench and is independent of the simulator; the Questa transcript may add compile and startup messages.

```
# =====
#           STARTING APB SUBSYSTEM TESTBENCH
# =====
# [STATUS] System Reset De-asserted.
#
# --- TEST 1: Full-Word Access (Slave 1) ---
# [WRITE SUCCESS] Addr: 0x00000000 | Data: 0xdeadbeef | Strobe: 4'b1111
# [WRITE SUCCESS] Addr: 0x00000004 | Data: 0xcafebabe | Strobe: 4'b1111
# [READ SUCCESS] Addr: 0x00000000 | Data Read: 0xxxxxxxxx
# [READ SUCCESS] Addr: 0x00000004 | Data Read: 0xdeadbeef
# -> TEST 1 PASSED!
#
# --- TEST 2: Byte-Strobe Masking (Slave 2) ---
# [WRITE SUCCESS] Addr: 0x00004000 | Data: 0x00000000 | Strobe: 4'b1111
# [WRITE SUCCESS] Addr: 0x00004000 | Data: 0xffff1234 | Strobe: 4'b0011
# [READ SUCCESS] Addr: 0x00004000 | Data Read: 0xxxxxxxxx
# --- TEST 3: Out-Of-Bounds PSLVERR Verification ---
# [WRITE SUCCESS] Addr: 0x00004000 | Data: 0xa5a5a5a5 | Strobe: 4'b1111
# --- TEST 4: Streaming Back-To-Back Transfers ---
# [WRITE SUCCESS] Addr: 0x00004008 | Data: 0x11112222 | Strobe: 4'b1111
```

```
# [WRITE SUCCESS] Addr: 0x0000400c | Data: 0x33334444 | Strobe: 4'b1111
# [READ SUCCESS] Addr: 0x00004008 | Data Read: 0x00001234
# [READ SUCCESS] Addr: 0x0000400c | Data Read: 0x11112222
# -> Back-To-Back Stream PASSED!
#
# =====
#          ALL APB SUBSYSTEM TESTS COMPLETED
# =====
# ** Note: $finish      : D:/Projects/AMBA APB/APB_Top_tb.v(204)
#      Time: 625 ns   Iteration: 1   Instance: /tb_apb_subsystem
```

8. Conclusion

The project provides a working APB4 subsystem structure with a master FSM, address decoder, two byte-writable memory slaves and a response multiplexer. Simulation confirms correct SETUP/ACCESS sequencing, slave selection and write handling. It also shows that the read path returns stale data and that the error test does not reach PSLVERR; these points are recorded in Section 9.2 together with proposed fixes and the extra tests that would complete the verification.

Appendix A: File List

File	Description
APB_Address_Decoder.v	Module apb_address_decoder.
APB_Master_Controller.v	Module apb_master_controller.
APB_Slave_Controller.v	Module apb_slave_controller.
APB_Top_Module.v	Module apb_subsystem_top: integration, select gating and response multiplexer.
APB_Top_tb.v	Testbench tb_apb_subsystem.
run.do	Questa / ModelSim script: compile, simulate, add waves and run.

Appendix B: Testbench and Script Listings

The listings of the design modules are given in Sections 5.1 to 5.4.

B.1 Testbench (APB_Top_tb.v)

```
`timescale 1ns / 1ps

module tb_apb_subsystem;

    // Parameters
    localparam DWIDTH      = 32;
    localparam ADDRWIDTH   = 32;
    localparam MEMDEPTH     = 4096;
    localparam CLK_PERIOD = 10; // 100 MHz Clock

    // Clock and Reset
```

```

reg pclk;
reg prstn;

// System Request Interface
reg req_valid;
reg req_write;
reg [ADDRWIDTH-1:0] req_addr;
reg [DWIDTH-1:0] req_wdata;
reg [3:0] req_strb;
reg [2:0] req_prot;

// System Response Interface
wire req_ready;
wire rsp_valid;
wire rsp_error;
wire busy;
wire done;
wire [DWIDTH-1:0] rsp_rdata;

// Output Storage for Tasks
reg [DWIDTH-1:0] read_data_buf;

// =====
// Device Under Test (DUT) Instantiation
// =====
apb_subsystem_top #(
    .DWIDTH      (DWIDTH),
    .ADDRWIDTH   (ADDRWIDTH),
    .MEMDEPTH    (MEMDEPTH)
) dut (
    .pclk        (pclk),
    .prstn       (prstn),
    .req_valid    (req_valid),
    .req_write    (req_write),
    .req_addr     (req_addr),
    .req_wdata    (req_wdata),
    .req_strb     (req_strb),
    .req_prot     (req_prot),
    .req_ready    (req_ready),
    .rsp_valid    (rsp_valid),
    .rsp_error    (rsp_error),
    .busy         (busy),
    .done         (done),
    .rsp_rdata    (rsp_rdata)
);

```

```

// =====
// Clock Generation
// =====

always #(CLK_PERIOD / 2) pclk = ~pclk;

// =====
// Helper Tasks for Bus Transactions
// =====

// Task: Issue APB Write Transfer
task apb_write(
    input [ADDRWIDTH-1:0] addr,
    input [DWIDTH-1:0] data,
    input [3:0] strb
);
    begin
        @(posedge pclk);
        while (!req_ready) @(posedge pclk); // Wait until Master is ready

        req_valid <= 1'b1;
        req_write <= 1'b1;
        req_addr <= addr;
        req_wdata <= data;
        req_strb <= strb;
        req_prot <= 3'b000;

        @(posedge pclk);
        req_valid <= 1'b0; // Clear valid after master accepts request

        // Wait for transaction completion
        while (!done) @(posedge pclk);

        if (rsp_error)
            $display("[WRITE ERROR] Address: 0x%08h flagged PSLVERR!", addr);
        else
            $display("[WRITE SUCCESS] Addr: 0x%08h | Data: 0x%08h | Strobe:
4'b%04b", addr, data, strb);
        end
    endtask

// Task: Issue APB Read Transfer
task apb_read(
    input [ADDRWIDTH-1:0] addr,
    output [DWIDTH-1:0] data
);
    begin

```

```

    @(posedge pclk);
    while (!req_ready) @(posedge pclk);

    req_valid <= 1'b1;
    req_write <= 1'b0; // Read operation
    req_addr <= addr;
    req_wdata <= 32'h0;
    req_strb <= 4'b0000;
    req_prot <= 3'b000;

    @(posedge pclk);
    req_valid <= 1'b0;

    // Wait for response completion
    while (!done) @(posedge pclk);

    data = rsp_rdata;
    if (rsp_error)
        $display("[READ ERROR] Address: 0x%08h flagged PSLVERR!", addr);
    else
        $display("[READ SUCCESS] Addr: 0x%08h | Data Read: 0x%08h", addr,
data);
    end
endtask

// =====
// Main Test Stimulus
// =====
initial begin
    // Initialize Inputs
    pclk      = 1'b0;
    prstn     = 1'b0;
    req_valid  = 1'b0;
    req_write  = 1'b0;
    req_addr   = {ADDRWIDTH{1'b0}};
    req_wdata  = {DWIDTH{1'b0}};
    req_strb   = 4'b0000;
    req_prot   = 3'b000;

    $display("\n=====");
    $display("          STARTING APB SUBSYSTEM TESTBENCH          ");
    $display("=====\\n");

    // 1. Reset Pulse
    #20;
    prstn = 1'b1;

```

```

$display("[STATUS] System Reset De-asserted.\n");
#10;

// -----
// TEST 1: Full-Word Access to Slave 1 (Base Addr: 0x0000_0000)
// -----
$display("--- TEST 1: Full-Word Access (Slave 1) ---");
apb_write(32'h0000_0000, 32'hDEAD_BEEF, 4'b1111);
apb_write(32'h0000_0004, 32'hCAFE_BABE, 4'b1111);

apb_read(32'h0000_0000, read_data_buf);

apb_read(32'h0000_0004, read_data_buf);
$display("-> TEST 1 PASSED!\n");

// -----
// TEST 2: Byte-Strobe Writes to Slave 2 (Base Addr: 0x0000_4000)
// -----
$display("--- TEST 2: Byte-Strobe Masking (Slave 2) ---");
// Step A: Initialize location with 0x0000_0000
apb_write(32'h0000_4000, 32'h0000_0000, 4'b1111);

// Step B: Write lower two bytes only (pstrb = 4'b0011)
apb_write(32'h0000_4000, 32'hFFFF_1234, 4'b0011);

// Step C: Verify top bytes remain 0x0000
apb_read(32'h0000_4000, read_data_buf);
if (read_data_buf === 32'h0000_1234)
    $display("-> Byte Strobe Verification PASSED! Expected: 0x00001234 | Got:
0x%08h\n", read_data_buf);

// -----
// TEST 3: Out-of-Bounds Address Detection (PSLVERR)
// -----
$display("--- TEST 3: Out-Of-Bounds PSLVERR Verification ---");
// Memory depth is 4096 words (0x3FFF max offset).
// Offset 0x4000 is 1 word past the boundary!
apb_write(32'h0000_3FFF + 32'h0000_0001, 32'hA5A5_A5A5, 4'b1111);
if (rsp_error)
    $display("-> PSLVERR correctly generated for Out-Of-Bounds address!\n");

// -----
// TEST 4: Back-To-Back Stream Transfers
// -----

```

```

$display("--- TEST 4: Streaming Back-To-Back Transfers ---");
apb_write(32'h0000_4008, 32'h1111_2222, 4'b1111);
apb_write(32'h0000_400C, 32'h3333_4444, 4'b1111);
apb_read(32'h0000_4008, read_data_buf);
apb_read(32'h0000_400C, read_data_buf);
$display("-> Back-To-Back Stream PASSED!\n");

$display("=====");
$display("      ALL APB SUBSYSTEM TESTS COMPLETED      ");
$display("=====");
$finish;
end

endmodule

```

References

- [1] ARM, AMBA APB Protocol Specification, Version 2.0, ARM IHI 0024C.
- [2] AMD Xilinx, Vivado Design Suite User Guide: Synthesis (UG901).